

# Face Recognition System — Project Documentation

## Overview

A real-time face recognition attendance system built on Raspberry Pi using Python. The system captures facial images, trains a recognition model, and identifies individuals through a live camera feed — all managed through a desktop GUI.

## Project Structure

```
Face/
├─ gui.py           # Main Tkinter GUI application
├─ capture.py       # Face capture module (Raspberry Pi Camera)
├─ train.py         # Model training module
├─ recog.py         # Real-time recognition module
├─ dataset/         # Captured face images per person
│   └─ <ntid>/      # e.g. dataset/johndoe/image_0.jpg
├─ encodings.pickle # Trained face encodings database
├─ user.json        # Attendance / login records
├─ admins.txt       # Registered admin NTID
├─ settings.json    # Encrypted credentials + server path
└─ LOG/            # Daily timestamped log files
    └─ YYYY-MM-DD.log
```

## Tech Stack & Dependencies

| Library               | Purpose                                  |
|-----------------------|------------------------------------------|
| face_recognition      | Face encoding & comparison (dlib HOG)    |
| picamera2             | Raspberry Pi Camera interface            |
| opencv-python         | Haar Cascade detection, image processing |
| tkinter               | Desktop GUI                              |
| cryptography (Fernet) | Credential encryption                    |
| requests              | SOAP API calls to Active Directory       |

| Library                | Purpose                                   |
|------------------------|-------------------------------------------|
| <code>pickle</code>    | Encoding serialisation                    |
| <code>threading</code> | Non-blocking camera & recognition threads |

## How It Works

### 1. Capture (`capture.py`)

- Opens Raspberry Pi Camera via `picamera2`
- Detects face in live frame using **Haar Cascade** (`haarcascade_frontalface_default.xml`)
- User clicks **Take Photo** in GUI to save the frame as `.jpg` into `dataset/<ntid>/`
- Supports `add` mode (append) and `replace` mode (overwrite existing photos)
- Camera retries up to 5 times if busy

### 2. Train (`train.py`)

- Scans `dataset/` folder for all person subfolders and images
- Converts each image to RGB, detects face location using **HOG model**
- Encodes face into a **128-dimensional vector** using `face_recognition.face_encodings()`
- Skips already-trained images using a `trained_files` key tracked in `encodings.pickle`
- Saves all encodings, names, and trained file list back to `encodings.pickle`

### 3. Recognize (`recog.py`)

- Opens Raspberry Pi Camera and reads frames via a background `FrameReader` thread
- Every 3rd frame is processed to reduce CPU load
- Detects face using **Haar Cascade**, then encodes using `face_recognition`
- Compares live encoding against stored encodings using **Euclidean distance**
- Match criteria: `tolerance=0.27` + `confidence >= 0.70`
- On match: updates `user.json`, writes to daily log, syncs log to PC server
- Cooldown: **1 minute** between repeated recognitions of the same person

## GUI (`gui.py`)

### Layout

- **Header** — App title, Login/Logout buttons, Settings (⚙) button
- **Left Panel** — Live camera feed + control buttons + NTID entry
- **Right Panel** — Dataset status per user + system log

### Buttons & Permissions

| Button         | Admin Only | Description                        |
|----------------|------------|------------------------------------|
| Capture Face   | ✓          | Start camera for face capture      |
| Take Photo     | ✓          | Trigger photo save during capture  |
| Train Model    | ✓          | Run incremental training           |
| Recognize Face | ✗          | Start live recognition (all users) |
| Stop           | ✗          | Stop any running process           |
| Delete User    | ✓          | Remove user dataset + encodings    |

### Role-Based Access Control

- On first login, the NTID is saved as admin in `admins.txt`
- Subsequent logins are checked against the saved admin NTID
- All NTIDs are validated against **Active Directory** via SOAP API before login or capture

### Active Directory Integration

- Endpoint: `http://jpetewebapp/jtesw_ws/jtesw_webservice.asmx`
- Method: `IsUserExistsInAD`
- Protocol: SOAP 1.2
- Returns `true/false` — parsed from XML response tag `ReturnedValue`
- Used in: **Login** and **Capture Face** (NTID must exist in AD)

## Network / Server Sync

- Mounts a Windows shared folder (CIFS/SMB) at `/mnt/pcshare` on startup
  - Mount command uses `sudo mount -t cifs` with domain credentials
  - **Settings popup** (⚙️) allows user to set NTID, password, and UNC path (e.g. `\\server\folder`)
  - Password is encrypted with **Fernet** before saving to `settings.json`
  - `user.json` is synced bidirectionally (local ↔ server) based on file modification time
  - Daily `.log` files are written both locally (`LOG/`) and to the server share
- 

## Settings (`settings.json`)

```
json
{
  "username": "your_ntid",
  "password": "<fernet_encrypted_string>",
  "pc_save_path": "\\server\\sharedfolder"
}
```

■ Password is encrypted using a fixed Fernet key (`FERNET_KEY`) defined in `gui.py`

---

## Attendance Log Format

**File:** `LOG/YYYY-MM-DD.log`

```
[2025-01-15 08:32:10] Recognized: johndoe | Confidence: 91.3%
[2025-01-15 09:01:45] Recognized: janesmith | Confidence: 87.6%
```

---

## user.json Format

```
json
```

```
{  
  "johndoe": {  
    "first_registered": "2025-01-10 08:00:00",  
    "last_login": "2025-01-15 08:32:10"  
  }  
}
```

---

## Known Constraints

- Only **1 face per image** is accepted during training (multi-face images are skipped)
  - Recognition processes every **3rd frame** to manage CPU load on Pi
  - Admin is **first-come-first-registered** — whoever logs in first becomes admin
  - Server mount requires `sudo` privileges on the Raspberry Pi
  - Virtual environment must be created with `--system-site-packages` to access `picamera2`
- 

## Virtual Environment Setup (Raspberry Pi)

```
bash  
  
python3 -m venv faceenv --system-site-packages  
source faceenv/bin/activate  
pip install face_recognition opencv-python cryptography requests
```

---

## Run the Application

```
bash  
  
cd ~/Face  
source faceenv/bin/activate  
python gui.py
```